

C++ Pattern Printing Programs

Programming Practice Assignment — Nested Loops

1. Square Pattern

Problem Statement: Print a square of stars with side length `n`.

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            cout << "* ";
        }
        cout << endl;
    }
    return 0;
}
```

Sample Output:

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

Logic Explanation: Both loops run a fixed `n` times. For every row (outer loop), the inner loop prints `n` stars, producing a perfect square grid.

Time Complexity: $O(n^2)$ — n rows, each with n iterations.

2. Hollow Square Pattern

Problem Statement: Print a square that has stars only on its border (first/last row and first/last column), with blank space inside.

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (i == 1 || i == n || j == 1 || j == n)
                cout << "*" << " ";
            else
                cout << "   ";
        }
        cout << endl;
    }
    return 0;
}
```

Sample Output:

```
* * * * *
*       *
*       *
*       *
*       *
* * * * *
```

Logic Explanation: A condition checks whether the current cell lies on the boundary (`i==1` , `i==n` , `j==1` , `j==n`). Boundary cells get `*` , all other cells get blank spaces, leaving the interior hollow.

Time Complexity: $O(n^2)$ — every cell of the $n \times n$ grid is visited once.

3. Right Triangle Pattern

Problem Statement: Print a right-angled triangle where row `i` contains `i` stars.

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            cout << "*" << " ";
        }
    }
}
```

```

        cout << endl;
    }
    return 0;
}

```

Sample Output:

```

*
* *
* * *
* * * *
* * * * *

```

Logic Explanation: The inner loop's upper bound depends on the outer loop variable `i`, so each successive row prints one more star than the previous row.

Time Complexity: $O(n^2)$ — total stars printed = $1+2+3+\dots+n = n(n+1)/2$, which is $O(n^2)$.

4. Inverted Triangle Pattern

Problem Statement: Print a triangle where the number of stars decreases from `n` to `1` row by row.

```

#include <iostream>
using namespace std;

int main() {
    int n = 5;
    for (int i = n; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            cout << "* ";
        }
        cout << endl;
    }
    return 0;
}

```

Sample Output:

```

* * * * *
* * * *
* * *

```

```
* *  
*
```

Logic Explanation: The outer loop decreases from `n` to `1`. The inner loop prints `i` stars for the current row, so the star count shrinks each iteration.

Time Complexity: $O(n^2)$ — sum of stars = $n + (n-1) + \dots + 1 = n(n+1)/2$.

5. Pyramid Pattern

Problem Statement: Print a centered pyramid of stars with `n` rows.

```
#include <iostream>  
using namespace std;  
  
int main() {  
    int n = 5;  
    for (int i = 1; i <= n; i++) {  
        for (int j = 1; j <= n - i; j++) {  
            cout << " ";  
        }  
        for (int k = 1; k <= i; k++) {  
            cout << "* ";  
        }  
        cout << endl;  
    }  
    return 0;  
}
```

Sample Output:

```
    *  
   * *  
  * * *  
 * * * *  
* * * * *
```

Logic Explanation: Two inner loops are used per row. The first prints `(n - i)` blank spaces (which decreases as `i` increases) to center the pyramid. The second prints `i` stars, which increases each row.

Time Complexity: $O(n^2)$ — both loops together cover $O(n)$ work per row across n rows.

6. Diamond Pattern

Problem Statement: Print a diamond shape made of stars (pyramid on top, inverted pyramid on bottom).

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n - i; j++)
            cout << " ";
        for (int k = 1; k <= 2 * i - 1; k++)
            cout << "* ";
        cout << endl;
    }

    for (int i = n - 1; i >= 1; i--) {
        for (int j = 1; j <= n - i; j++)
            cout << " ";
        for (int k = 1; k <= 2 * i - 1; k++)
            cout << "* ";
        cout << endl;
    }

    return 0;
}
```

Sample Output:

```
      *
     * * *
    * * * * *
   * * * * * * *
  * * * * * * * *
 * * * * * * * *
* * * * * * *
 * * * * *
  * * * *
   * * *
    * *
     *
```

Logic Explanation: The upper half is a pyramid where row i prints $2*i-1$ stars (1, 3, 5, 7, 9...) with $(n-i)$ leading spaces. The lower half mirrors this in reverse (from $n-1$ down to 1), creating the bottom half of the diamond.

Time Complexity: $O(n^2)$ — two pyramid-like loops, each $O(n^2)$.

7. Half Diamond Pattern

Problem Statement: Print a left-aligned half diamond (increasing triangle followed by decreasing triangle, no centering spaces).

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++)
            cout << "* ";
        cout << endl;
    }

    for (int i = n - 1; i >= 1; i--) {
        for (int j = 1; j <= i; j++)
            cout << "* ";
        cout << endl;
    }

    return 0;
}
```

Sample Output:

```
*
* *
* * *
* * * *
* * * * *
* * * * *
* * * *
* * *
* *
*
```

Logic Explanation: The first block prints rows with 1 to n stars (increasing), and the second block prints rows with n-1 down to 1 stars (decreasing), both left-aligned with no spaces.

Time Complexity: $O(n^2)$ — combination of two right-triangle loops, each $O(n^2)$.

8. Inverted Half Diamond Pattern

Problem Statement: Print the inverted version of the half diamond (decreasing triangle followed by increasing triangle).

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;

    for (int i = n; i >= 1; i--) {
        for (int j = 1; j <= i; j++)
            cout << "* ";
        cout << endl;
    }

    for (int i = 2; i <= n; i++) {
        for (int j = 1; j <= i; j++)
            cout << "* ";
        cout << endl;
    }

    return 0;
}
```

Sample Output:

```
* * * * *
* * * *
* * *
* *
*
* *
* * *
* * * *
* * * * *
```

Logic Explanation: The first block decreases the star count from `n` to `1`, and the second block increases it again from `2` to `n`, producing an inverted diamond outline.

Time Complexity: $O(n^2)$ — combination of two right-triangle loops, each $O(n^2)$.

9. Number Square Pattern

Problem Statement: Print a square where every value in a row equals the row number.

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            cout << i << " "; // print row number repeatedly
        }
        cout << endl;
    }
    return 0;
}
```

Sample Output:

```
1 1 1 1 1
2 2 2 2 2
3 3 3 3 3
4 4 4 4 4
5 5 5 5 5
```

Logic Explanation: The inner loop runs `n` times but always prints the outer loop's current value `i`, so the entire row repeats the same number.

Time Complexity: $O(n^2)$ — $n \times n$ grid, one print operation per cell.

10. Incrementing Number Square Pattern

Problem Statement: Print a square where each row contains the numbers 1 to n in order.

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            cout << j << " "; // print column index
        }
    }
}
```



```

        cout << endl;
    }
    return 0;
}

```

Sample Output:

```

1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5

```

Logic Explanation: The inner loop variable `j` (ranging from 1 to `n`) is printed directly, so every row reproduces the same incrementing sequence 1 through `n`.

Time Complexity: $O(n^2)$ — `n` x `n` grid, one print operation per cell.

11. Number Right Triangle Pattern

Problem Statement: Print a right triangle where row `i` contains the numbers 1 through `i`.

```

#include <iostream>
using namespace std;

int main() {
    int n = 5;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            cout << j << " ";
        }
        cout << endl;
    }
    return 0;
}

```

Sample Output:

```

1
1 2
1 2 3

```

```
1 2 3 4
1 2 3 4 5
```

Logic Explanation: The inner loop runs from `1` to `i` and prints `j` each time, so row `i` shows the sequence `1, 2, ..., i`.

Time Complexity: $O(n^2)$ — total elements printed = $1+2+\dots+n = n(n+1)/2$, which is $O(n^2)$.

12. Inverted Number Diamond Pattern

Problem Statement: Print a diamond shape using numbers, where the row width decreases then increases, and each row always starts counting from 1.

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;

    for (int i = n; i >= 1; i--) {
        for (int s = 1; s <= n - i; s++)
            cout << " ";
        for (int j = 1; j <= i; j++)
            cout << j << " ";
        cout << endl;
    }

    for (int i = 2; i <= n; i++) {
        for (int s = 1; s <= n - i; s++)
            cout << " ";
        for (int j = 1; j <= i; j++)
            cout << j << " ";
        cout << endl;
    }

    return 0;
}
```

Sample Output:

```
1 2 3 4 5
 1 2 3 4
  1 2 3
   1 2
    1
```

```
    1 2
   1 2 3
  1 2 3 4
 1 2 3 4 5
```

Logic Explanation: The upper half starts with `i = n` (no spaces, full sequence 1 to n) and decreases `i` while adding leading spaces, shrinking the printed sequence each row. The lower half reverses this from `i = 2` back to `n`, growing the sequence again — forming an inverted diamond made of numbers.

Time Complexity: $O(n^2)$ — spacing and number-printing loops together cover $O(n)$ work per row across $2n-1$ rows.

13. Sandwich Pattern

Problem Statement: Print a square where the border is made of `*` and the interior is filled with `0`, resembling a “sandwich” (filled crust, different filling inside).

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (i == 1 || i == n || j == 1 || j == n)
                cout << "*" << " ";
            else
                cout << "0" << " ";
        }
        cout << endl;
    }
    return 0;
}
```

Sample Output:

```
* * * * *
* 0 0 0 *
* 0 0 0 *
* 0 0 0 *
* * * * *
```

Logic Explanation: This uses the same border-detection condition as the hollow square, but instead of leaving the interior blank, it fills it with `0`, creating two distinct “layers” — the outer crust (`*`) and the inner filling (`0`).

Time Complexity: $O(n^2)$ — every cell of the $n \times n$ grid is checked and printed once.

14. Incrementing Diamond Pattern

Problem Statement: Print a diamond shape where, instead of stars, each row contains numbers incrementing from 1 up to the row's width.

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;

    for (int i = 1; i <= n; i++) {
        for (int s = 1; s <= n - i; s++)
            cout << " ";
        for (int j = 1; j <= 2 * i - 1; j++)
            cout << j << " ";
        cout << endl;
    }

    for (int i = n - 1; i >= 1; i--) {
        for (int s = 1; s <= n - i; s++)
            cout << " ";
        for (int j = 1; j <= 2 * i - 1; j++)
            cout << j << " ";
        cout << endl;
    }

    return 0;
}
```

Sample Output:

```
    1
   1 2 3
  1 2 3 4 5
 1 2 3 4 5 6 7
1 2 3 4 5 6 7 8 9
 1 2 3 4 5 6 7
   1 2 3 4 5
```

```
1 2 3
  1
```

Logic Explanation: This follows the same shape logic as the star Diamond Pattern (pattern 6), but the innermost loop prints numbers `1` through `2*i-1` instead of stars. Leading spaces `(n-i)` keep the diamond centered.

Time Complexity: $O(n^2)$ — same structure as the diamond pattern, with $O(n)$ work per row across $2n-1$ rows.

Summary Table

#	Pattern Name	Type	Time Complexity
1	Square Pattern	Star	$O(n^2)$
2	Hollow Square Pattern	Star	$O(n^2)$
3	Right Triangle Pattern	Star	$O(n^2)$
4	Inverted Triangle Pattern	Star	$O(n^2)$
5	Pyramid Pattern	Star	$O(n^2)$
6	Diamond Pattern	Star	$O(n^2)$
7	Half Diamond Pattern	Star	$O(n^2)$
8	Inverted Half Diamond Pattern	Star	$O(n^2)$
9	Number Square Pattern	Number	$O(n^2)$
10	Incrementing Number Square Pattern	Number	$O(n^2)$
11	Number Right Triangle Pattern	Number	$O(n^2)$
12	Inverted Number Diamond Pattern	Number	$O(n^2)$
13	Sandwich Pattern	Number/Star	$O(n^2)$
14	Incrementing Diamond Pattern	Number	$O(n^2)$

General Note: All programs use `n = 5` for demonstration. Change the value of `n` to print patterns of any size. All solutions use only nested `for` loops as required, with $O(n^2)$ time complexity since each cell of an $n \times n$ (or similarly bounded) grid is visited a constant number of times.